C++ PROGRAMMING LAB MANUAL

Unit I to Unit III

F.Y. C.S.E. | School of Computing
MIT Art, Design & Technology University, Pune

UNIT I - Introduction to Object-Oriented Programming

Experiment 1: Student Class with Details and Result Calculation

Objective

Create a Student class to accept roll number, name and marks, calculate total/percentage/result and display details.

Theory

A class combines data and functions. Objects are instances of a class. Arrays can store several marks inside each object.

Simple C++ Program

```
#include <iostream>
#include <string>
using namespace std;
class Student{int roll;string name;float m[3],total,per;public:void
accept(){cin>>roll;cin.ignore();getline(cin,name);for(int i=0;i<3;i++)cin>>m[i];}void calc(){total=0;for(int
i=0;i<3;i++)total+=m[i];per=total/3;}void display(){cout<<"Roll: "<<roll<<"\nName: "<<name<<"\nTotal:
"<<total<<"\nPercentage: "<<per<<"%\n";if(per>=60)cout<<"First Class";else if(per>=50)cout<<"Second
Class";else if(per>=40)cout<<"Pass";else cout<<"Fail";}};int main(){Student s;s.accept();s.calc();s.display();}
```

Sample Output

```
101
John Doe
85 78 92
Roll: 101
Name: John Doe
Total: 255
```

SWARNIMA KARMORE | 73 | SOC-2
Percentage: 85%
First Class

Solved Viva Questions

1. What is a class? How is it different from a structure?

A class is a user-defined type containing data and functions. Class members are private by default; structure members are public by default.

2. What is the purpose of access specifiers?

They control visibility of class members. private restricts outside access; public makes members accessible through objects.

3. Difference between cin and getline()?

cin normally stops at whitespace; getline() reads the whole line.

4. Why use cin.ignore()?

To remove the pending newline before getline().

5. What is an array of objects?

An array containing multiple objects of the same class, e.g. Student s[5].

6. Default access specifier in a class?

private.

7. How does this pointer work?

this points to the current object inside a non-static member function.

8. Difference between class and object?

A class is a blueprint; an object is an instance.

Experiment 2: Rectangle Class with Member Functions

Objective

Calculate rectangle area and perimeter using private data and member functions.

Theory

Functions defined outside a class use the scope resolution operator (::). Functions defined inside are implicitly inline.

Simple C++ Program

```
#include <iostream>
using namespace std;
class Rectangle{float l,b;public:void set(float x,float y){l=x;b=y;}float area();float perimeter();};float
```

```
Rectangle::area(){return l*b;}float Rectangle::perimeter(){return 2*(l+b);}int main(){Rectangle
r;r.set(10,5);cout<<"Area: "<<r.area()<<"\nPerimeter: "<<r.perimeter();}
```

Sample Output

```
Area: 50
Perimeter: 30
```

Solved Viva Questions

1. Inside vs outside class functions?

Inside definitions are implicitly inline; outside definitions separate interface and implementation and use class scope.

2. What is ::?

The scope resolution operator identifies the class or scope a name belongs to.

3. What is an inline function?

A function the compiler may expand at its call site.

4. Why keep data private?

To protect state and control access.

5. What is encapsulation?

Bundling data and functions together while restricting direct access.

6. How access private members from outside?

Through public member functions or declared friends.

7. Purpose of public?

It exposes members as the class interface.

8. Can an outside member definition omit ::?

Not when defining a class member in global scope; the class scope must be specified.

Experiment 3: Product Class with Array of Objects

Objective

Store products as an array of objects, accept 12 monthly sales and compute quantity, bill and grand total.

Theory

An array of objects stores multiple independent instances of the same class.

Simple C++ Program

SWARNIMA KARMORE | 73 | SOC-2

```
#include <iostream>
#include <string>
using namespace std;
class Product{int id,s[12],total;string name;float price;public:void
accept(){cin>>id;cin.ignore();getline(cin,name);cin>>price;total=0;for(int
i=0;i<12;i++){cin>>s[i];total+=s[i];}}float bill()const{return total*price;}void show()const{cout<<id<<"
"<<name<<" "<<total<<" "<<bill()<<endl;}};int main(){Product p[2];float g=0;for(int
i=0;i<2;i++)p[i].accept();for(int i=0;i<2;i++){p[i].show();g+=p[i].bill();}cout<<"Grand Total: "<<g;}
```

Sample Output

Example: two products are displayed with their total quantities and bills.
Grand Total: sum of both product bills.

Solved Viva Questions

1. How create an array of objects?

Use ClassName arrayName[size], e.g. Product p[5].

2. Array of objects vs array within class?

The first stores many complete objects; the second is one array data member in each object.

3. Memory allocation for array of objects?

Elements occupy contiguous storage and each element is constructed separately.

4. What is a static data member?

One class variable shared by all objects.

5. Use of const member functions?

They do not modify object state and can be called on const objects.

6. Initialize array of objects using constructors?

Use an initializer list containing constructor calls.

7. Stack vs heap allocation?

Automatic local objects have scope-based lifetime; dynamic objects use allocated storage and need lifetime management.

8. Dynamic array of objects?

Prefer vector<Product>; raw arrays can use new[] and delete[].

UNIT II - Classes and Constructors

Experiment 1: Employee Class with Constructors

Objective

Use default and parameterized constructors and calculate salary with bonus.

Theory

Constructors initialize objects automatically. Constructor overloading permits different parameter lists.

Simple C++ Program

```
#include <iostream>
#include <string>
using namespace std;class Employee{int id;string name;float
sal,bonus;public:Employee(){id=0;name="Unknown";sal=bonus=0;}Employee(int i,string n,float s,float
b){id=i;name=n;sal=s;bonus=b;}void show(){cout<<id<<" "<<name<<" "<<sal+bonus<<endl;}};int
main(){Employee e1;Employee e2(101,"John Doe",50000,10000);e1.show();e2.show();}
```

Sample Output

```
0 Unknown 0
101 John Doe 60000
```

Solved Viva Questions

1. What is a constructor?

A special member automatically called when an object is created to initialize it.

2. Default vs parameterized constructor?

Default can be called without arguments; parameterized receives initialization values.

3. Constructor overloading?

Multiple constructors with different parameter lists.

4. Copy constructor?

A constructor that initializes a new object from another object of the same class.

5. Purpose of initialization list?

It initializes base classes and data members directly before the body.

6. What is a destructor?

A special cleanup function called automatically when an object is destroyed.

7. Can a constructor be private?

Yes, to restrict or control object creation.

8. Constructor vs member function?

A constructor initializes objects and has no return type; ordinary member functions perform operations.

Experiment 2: Book Class with Copy Constructor

Objective

Create a Book using a parameterized constructor and copy it using a copy constructor.

Theory

A copy constructor creates a new object from an existing object. Deep copying is needed when independent owned resources must be duplicated.

Simple C++ Program

```
#include <iostream>
#include <string>
using namespace std;class Book{string title,author;float price;public:Book(string t,string a,float p){title=t;author=a;price=p;}Book(const Book& b){title=b.title;author=b.author;price=b.price;}void show(){cout<<title<<" | "<<author<<" | "<<price<<endl;}};int main(){Book b1("C++ Basics","A. Kumar",450);Book b2=b1;b1.show();b2.show();}
```

Sample Output

```
C++ Basics | A. Kumar | 450
C++ Basics | A. Kumar | 450
```

Solved Viva Questions

1. Shallow vs deep copy?

Shallow copy duplicates member values such as pointer addresses; deep copy duplicates the owned resource itself.

2. When is copy constructor called?

When a new object is initialized from an existing object, subject to copy elision.

3. Copy constructor vs assignment?

Copy construction creates a new object; assignment changes an existing one.

4. Why is destructor needed?

To release owned resources during object destruction.

5. Rule of three/five?

Resource-managing classes often need destructor, copy constructor and copy assignment; modern C++ adds move constructor and move assignment.

6. Move constructor?

It transfers resources from an expiring object instead of copying them.

7. If no copy constructor is defined?

The compiler may generate member-wise copying, which can be unsafe for raw owned resources.

8. Use of static in a class?

Static data is shared among objects; static functions belong to the class.

Experiment 3: Static Members and Friend Functions

Objective

Demonstrate shared static members and friend access.

Theory

Static members belong to the class as a whole. Friends are explicitly granted access to private/protected members.

Simple C++ Program

```
#include <iostream>
using namespace std;class Counter{static int count;public:Counter(){count++;}static void
show(){cout<<"Objects: "<<count<<endl;}};int Counter::count=0;class B;class A{int x;public:A(int
v):x(v){}friend void cmp(A,B);};class B{int y;public:B(int v):y(v){}friend void cmp(A,B);};void cmp(A a,B
b){if(a.x>b.y)cout<<"A is greater";else if(a.x<b.y)cout<<"B is greater";else cout<<"Equal";}int main(){Counter
a,b,c;Counter::show();A x(50);B y(30);cmp(x,y);}
```

Sample Output

```
Objects: 3
A is greater
```

Solved Viva Questions

1. What is a static data member?

A variable shared by all objects of a class.

2. What is a static member function?

A class-level function callable without a specific object.

3. Why can’t static functions access non-static members directly?

They have no this pointer/current object.

4. What is a friend function?

A non-member granted access to private/protected members.

5. Friend vs member function?

A member belongs to the class and has this; a friend is external but has granted access.

6. What is a friend class?

A class permitted to access another class's private/protected members.

7. Is friendship inherited?

No.

8. Advantages/disadvantages of friend functions?

They are convenient for tightly related operations but weaken encapsulation and can increase coupling.

UNIT III - Inheritance

Experiment 1: Single and Multilevel Inheritance

Objective

Implement Person -> Student single inheritance and Vehicle -> Car -> ElectricCar multilevel inheritance.

Theory

Inheritance lets a derived class reuse and extend accessible members of a base class. Multilevel inheritance forms a chain.

Simple C++ Program

```
#include <iostream>
#include <string>
using namespace std;class Person{protected:string name;int age;public:Person(string n,int a){name=n;age=a;}};class Student:public Person{int roll;float marks;public:Student(string n,int a,int r,float m):Person(n,a){roll=r;marks=m;}void show(){cout<<name<<" "<<age<<" "<<roll<<" "<<marks<<endl;}};class Vehicle{protected:string brand;public:Vehicle(string b){brand=b;}};class Car:public Vehicle{protected:string model;public:Car(string b,string m):Vehicle(b){model=m;}};class ElectricCar:public Car{int battery;public:ElectricCar(string b,string m,int x):Car(b,m){battery=x;}void show(){cout<<brand<<" "<<model<<" "<<battery<<" kWh"<<endl;}};int main(){Student s("John Doe",20,101,85.5);s.show();ElectricCar e("Tesla","Model S",100);e.show();}
```

Sample Output

John Doe 20 101 85.5
Tesla Model S 100 kWh

Solved Viva Questions

1. What is inheritance? Why useful?

It allows a derived class to reuse and extend a base class, supporting reuse and hierarchies.

2. Public, protected and private inheritance?

Public keeps base public/protected as public/protected; protected makes them protected; private makes them private in the derived class.

3. Constructor execution order?

Base constructor first, then derived constructor. Destruction is reverse.

4. Single vs multilevel inheritance?

Single has one direct base; multilevel has a chain of derived classes.

5. What is method overriding?

A derived class provides its own implementation of a matching base-class function.

6. Inheritance vs composition?

Inheritance is is-a; composition is has-a.

7. Can constructor be inherited?

Not like an ordinary member, though C++ can make base constructors available with a using-declaration.

8. What is the diamond problem?

Two inheritance paths reach the same base, causing duplication/ambiguity; virtual inheritance can solve it.